Microsoft IQ Deep Dive with Python

July 28: Foundry IQ

July 29: Work IQ

July 30: Fabric IQ

Register at aka.ms/IQDeepDivePython/series

Microsoft IQ Deep Dive:
Work IQ

aka.ms/iqdeepdive/slides/workiq

Ayça Baş
Senior Cloud Advocate
Microsoft / GitHub
github.com/aycabas

Today we'll cover...

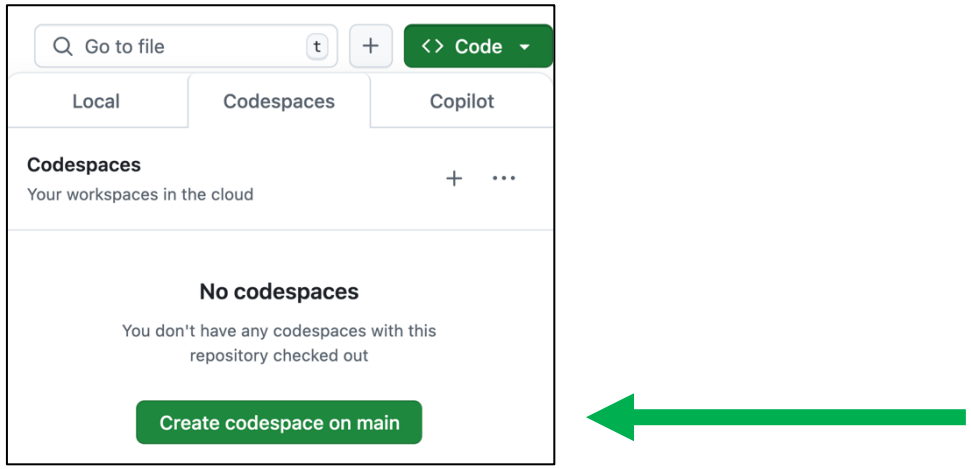
1. Work IQ overview
2. API concepts & protocols (A2A · MCP · REST)
3. A2A integration patterns
4. MCP integration & do_action (the only write path)
5. Building with Work IQ tools
6. Work IQ in a Microsoft Agent Framework agent
7. Ship it as an Agent 365 autopilot

Want the code?

1. Open this GitHub repository:

https://aka.ms/iqdeepdive

2. Use "Code" button to create a GitHub Codespace:



3. Wait a few minutes for Codespace to start up ⏱️

⚠️ Most code samples require deployment and an Azure account.

Microsoft IQ

Microsoft IQ Platform

Unified intelligence for enterprise AI

Work IQ

How your employees work

Context on people, collaboration, and workflows

Covering today: Session 2

Fabric IQ

How your business operates

Context on business entities, systems of record, and actions

Deep dive: Session 3

Web IQ

How you connect to web intelligence

Context from the web, news, images and video

Foundry IQ

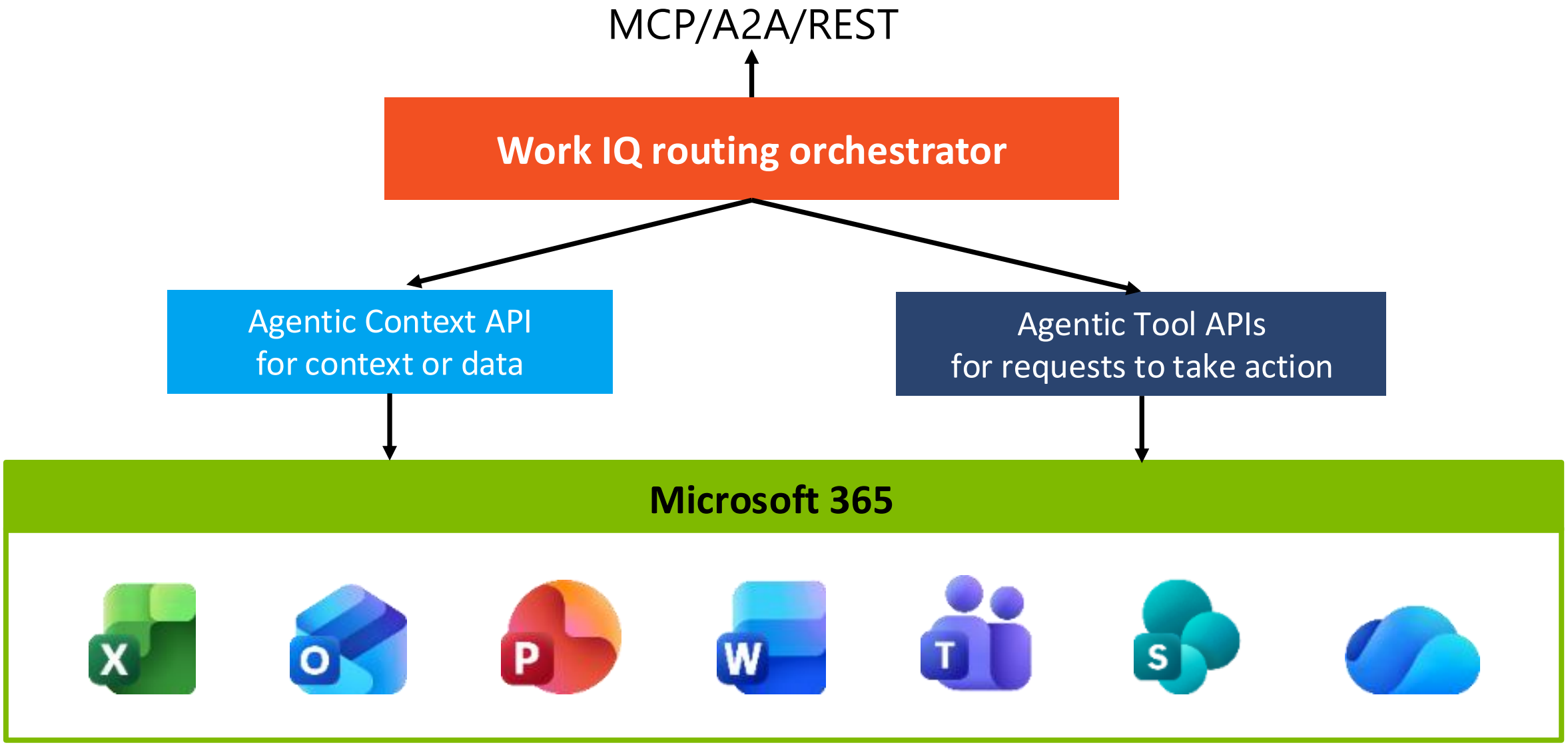
How your agents unlock knowledge

Context on policies, authoritative documents, and knowledge bases

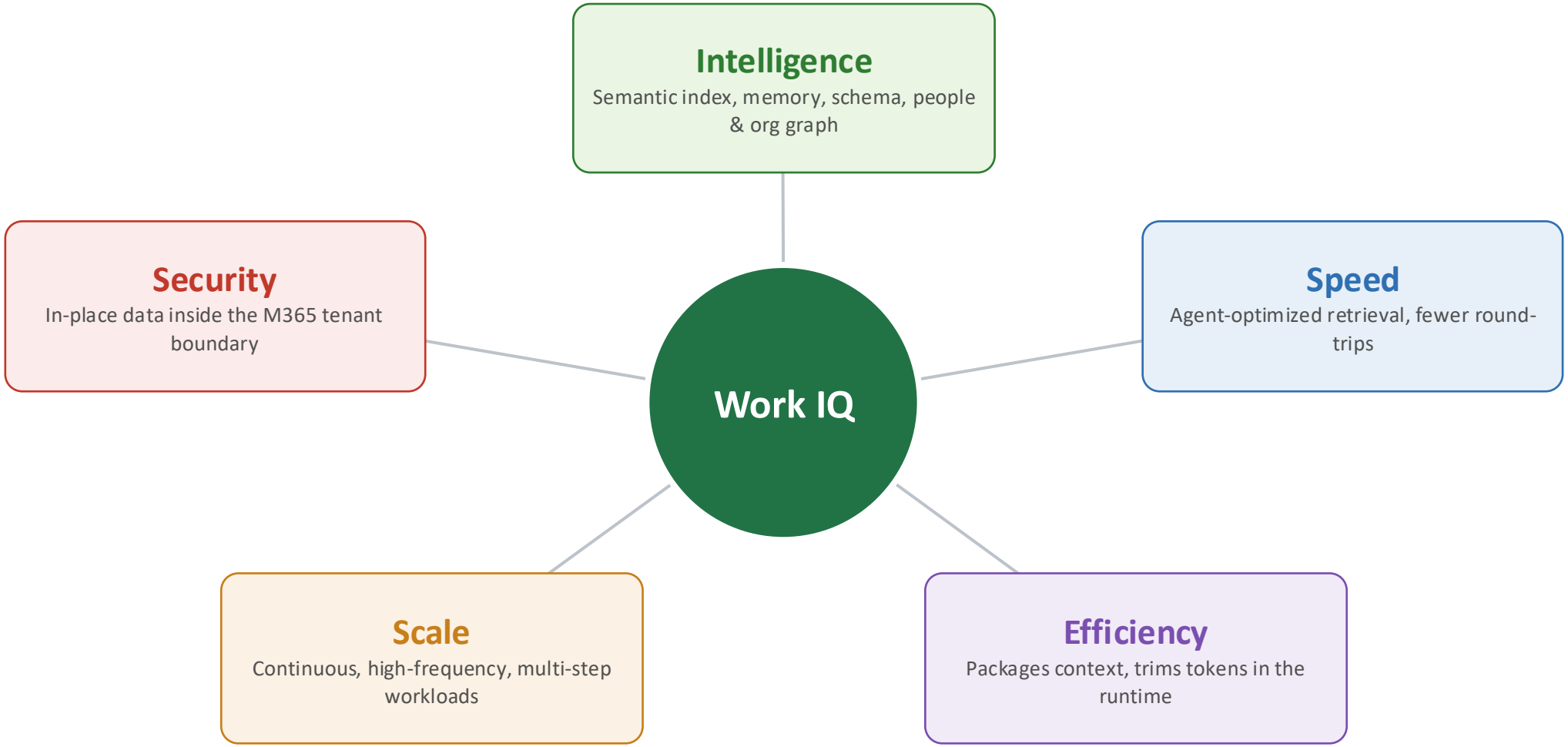
Deep dive: Session 1

Work IQ

Work IQ: An agent for work data



Why Work IQ?


Intelligence
Semantic index, memory, schema, people & org graph
Security
In-place data inside the M365 tenant boundary
Work IQ
Speed
Agent-optimized retrieval, fewer round-trips
Scale
Continuous, high-frequency, multi-step workloads
Efficiency
Packages context, trims tokens in the runtime

Work IQ vs Microsoft Graph

Microsoft Graph	Work IQ
A data-access API	An intelligence & context layer
Built for apps and users	Built for agents
App-only and delegated auth	Delegated (signed-in user) only
Returns raw data you stitch together	Returns grounded answers + references
Hundreds of typed endpoints	10 generic tools over MCP
You manage paging and throttling	Agent-optimized, in-place governance

The Work IQ API surface

Chat

Converse with people & other agents — ask, A2A

Context

Grounded org understanding across mail, meetings, docs & chats

Tools

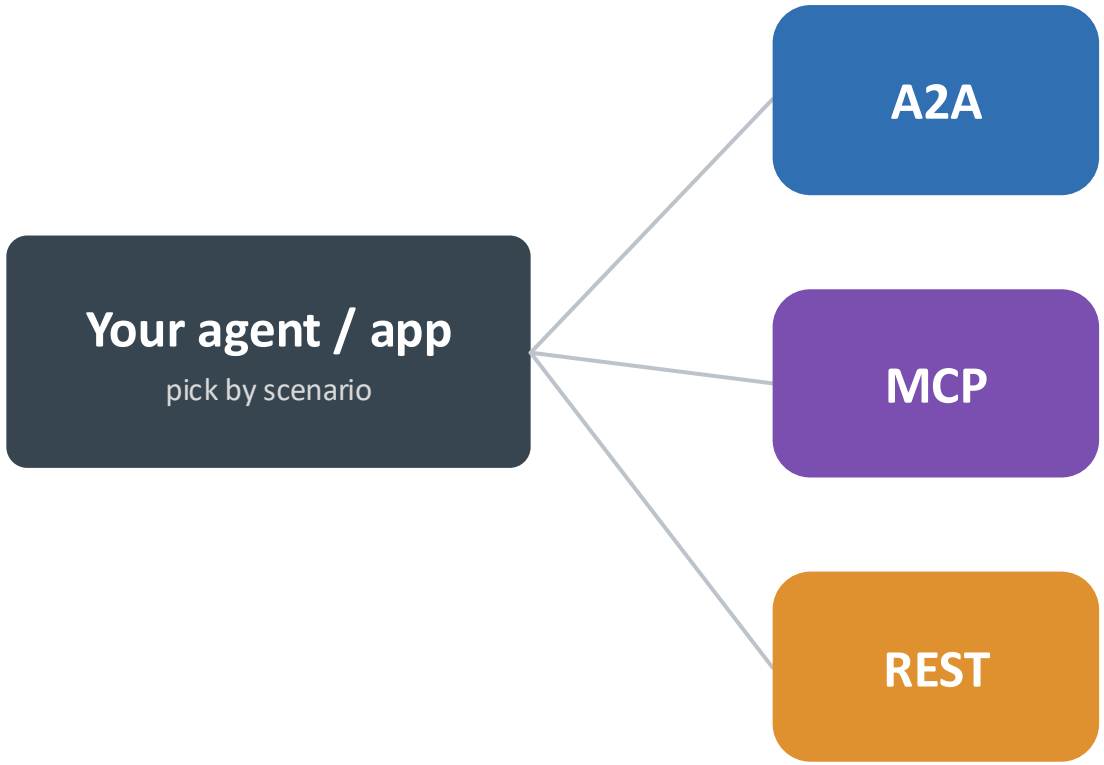
Governed retrieve + act across Microsoft 365 via generic verbs

Workspaces

A persistent place for agents to hold state across long tasks

One intelligence layer behind three protocols: A2A · MCP · REST

Protocol strategy: when to use A2A, MCP, REST



Agent-to-Agent

One agent delegates a task to another (multi-agent orchestration)

Agent-to-Tool

An agent grounds & acts via Work IQ as a tool

Human / device-to-Agent

A mobile or web app queries Work IQ for a user

Retrieval with Work IQ

Question

What did my manager email me about this week?

→

Work IQ

Source:	Outlook
People:	Priya Sharma

→

Results

Summary:	Priya sent 3 emails about the Q3 budget review: she needs your headcount numbers by Friday and moved the team sync to Thursday 2pm. [1][2][3]
Attributions:	https://outlook.office365.com/... https://outlook.office365.com/... https://outlook.office365.com/...

Building with Work IQ

Work IQ retrieval with ask

```
from notebooks._shared import get_user_token, ask

token = get_user_token()

# `ask` is a Work IQ MCP tool over /mcp - no REST chat route.
resp = ask(token,
    "Summarize my 5 most recent emails - who sent "
    "each and what they need from me.")

print(resp["result"]["structuredContent"]["answer"])
```

🔗 Full source: part1-workiq-api-concepts.ipynb

A2A agent card: how agents discover Work IQ

```
GET {gateway}/a2a/.well-known/agent-card.json
{
  "name": "Work IQ Relay Agent",
  "description": "Relays messages to Microsoft 365 Copilot",
  "protocolVersion": "0.3.0",
  "preferredTransport": "JSONRPC",
  "capabilities": { "streaming": true },
  "skills": [{
    "id": "ask_work_iq",
    "name": "Ask Work IQ",
    "examples": ["What meetings do I have today?",
                 "Summarize my recent emails from my manager"]
  }]
}
```

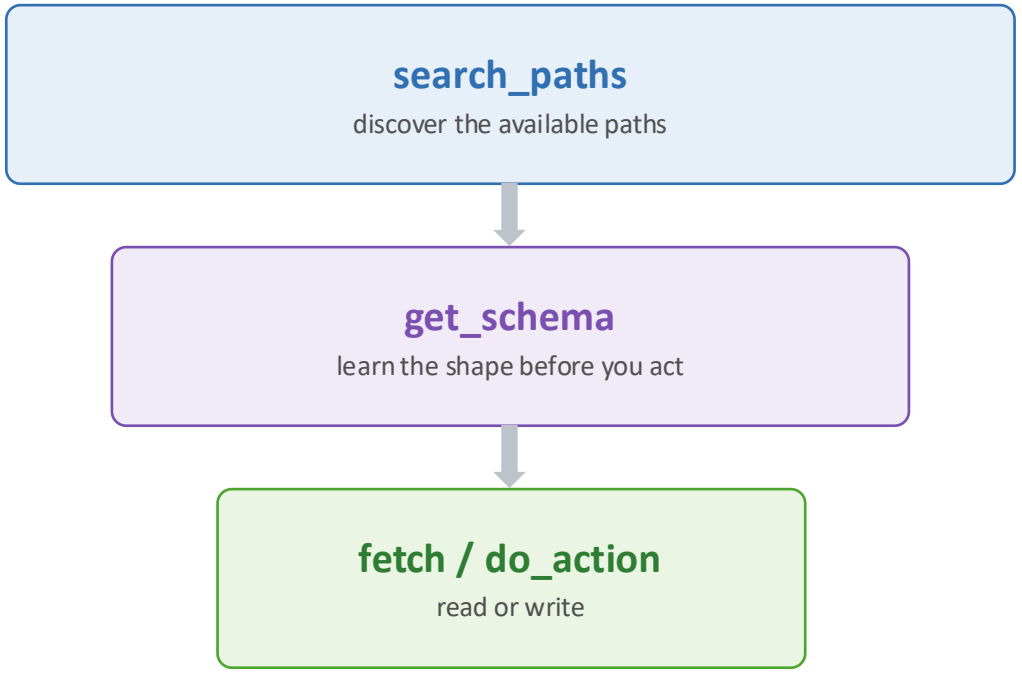
🔗 Discovery: GET /a2a/.well-known/agent-card.json

Work IQ via the A2A protocol

```
# Work IQ publishes a standard A2A agent card + message/send.
rpc = {"jsonrpc": "2.0", "id": "1", "method": "message/send",
  "params": {"message": {"kind": "message", "role": "user",
    "parts": [{"kind": "text", "text": "My recent Teams chats?"}],
    "messageId": "m1"}}}

r = requests.post(f"{WORK_IQ_GATEWAY}/a2a/", json=rpc,
  headers={"Authorization": f"Bearer {token}",
    "Accept": "application/json, text/event-stream"})

# Reply streams over SSE as a `task` whose artifacts hold the text.
task = parse_sse_task(r.text)
```

Full source: part2-workiq-a2a.ipynb

10 generic tools, progressive disclosure



10 generic tools

Chat
ask · list_agents

Schema
search_paths · get_schema

Entity read
fetch · call_function

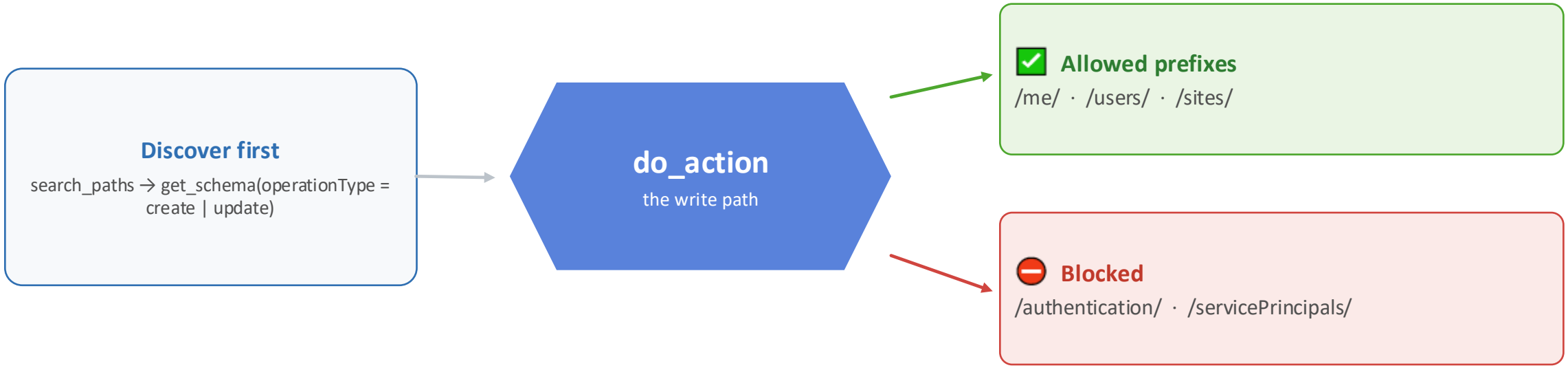
Entity write
create · update · delete · do_action

Work IQ via the MCP protocol

```
from notebooks._shared import get_user_token, call_mcp, call_tool

token = get_user_token()

# Work IQ speaks MCP (JSON-RPC 2.0) at {gateway}/mcp over SSE.
tools = call_mcp(token, "tools/list", {})
for t in tools["result"]["tools"]:
    print(t["name"], t["inputSchema"].get("required", []))

# fetch reads M365 entities by path via `entityUrls`.
msgs = call_tool(token, "fetch",
    {"entityUrls": ["/me/messages?$top=5&$select=subject,from"]})
```

Full source: part3-workiq-mcp.ipynb

The write path & its guardrails



Runs under the signed-in user · **jsonBody is a JSON-encoded string**

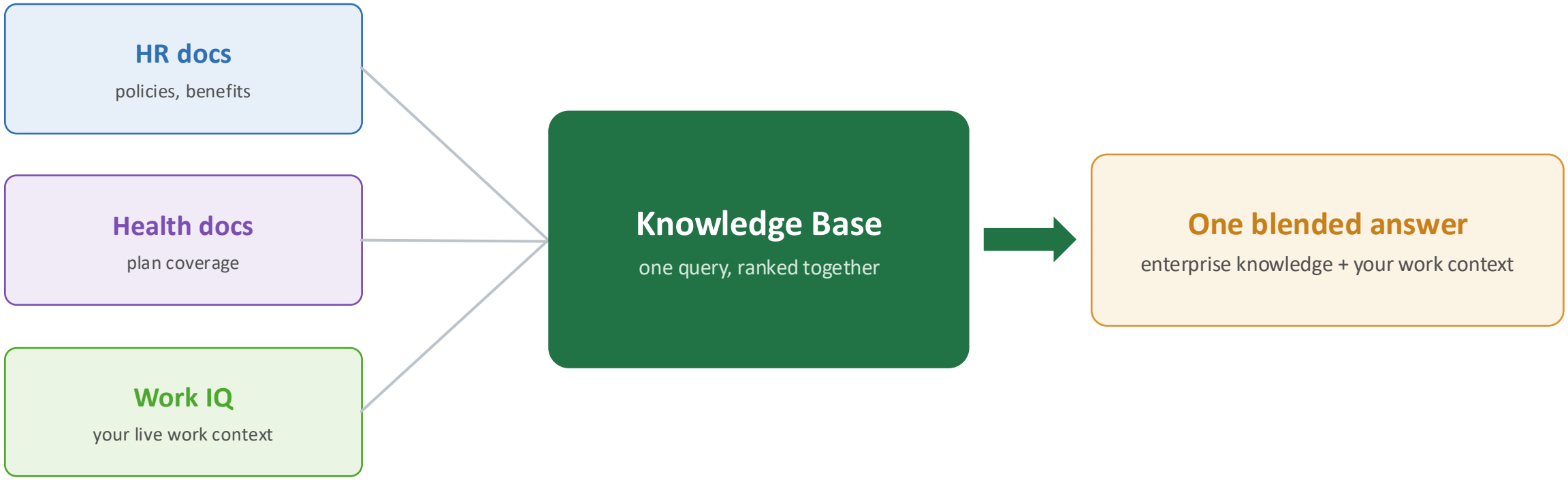
Taking action with do_action

```
# do_action is the ONLY write path (jsonBody = JSON string).
action_url = "/me/sendMail"
json_body = json.dumps({
  "Message": {
    "subject": "Re: your note",
    "body": {"contentType": "Text", "content": "<approved draft>"},
    "toRecipients": [
      {"emailAddress": {"address": "manager@contoso.com"}}],
  },
  "SaveToSentItems": True,
})
call_tool(token, "do_action",
  {"actionUrl": action_url, "jsonBody": json_body})
```

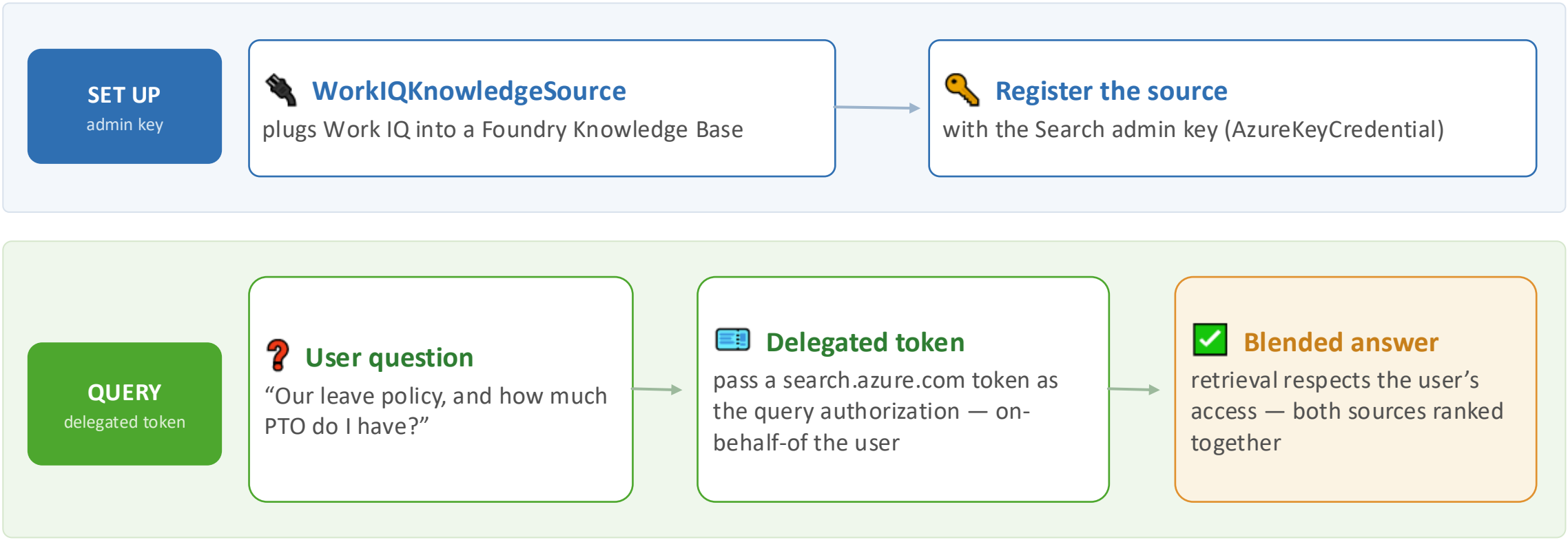
Full source: part4-workiq-tools-actions.ipynb

Foundry IQ + Work IQ

Foundry IQ: One knowledge base, every source



Work IQ as a knowledge source



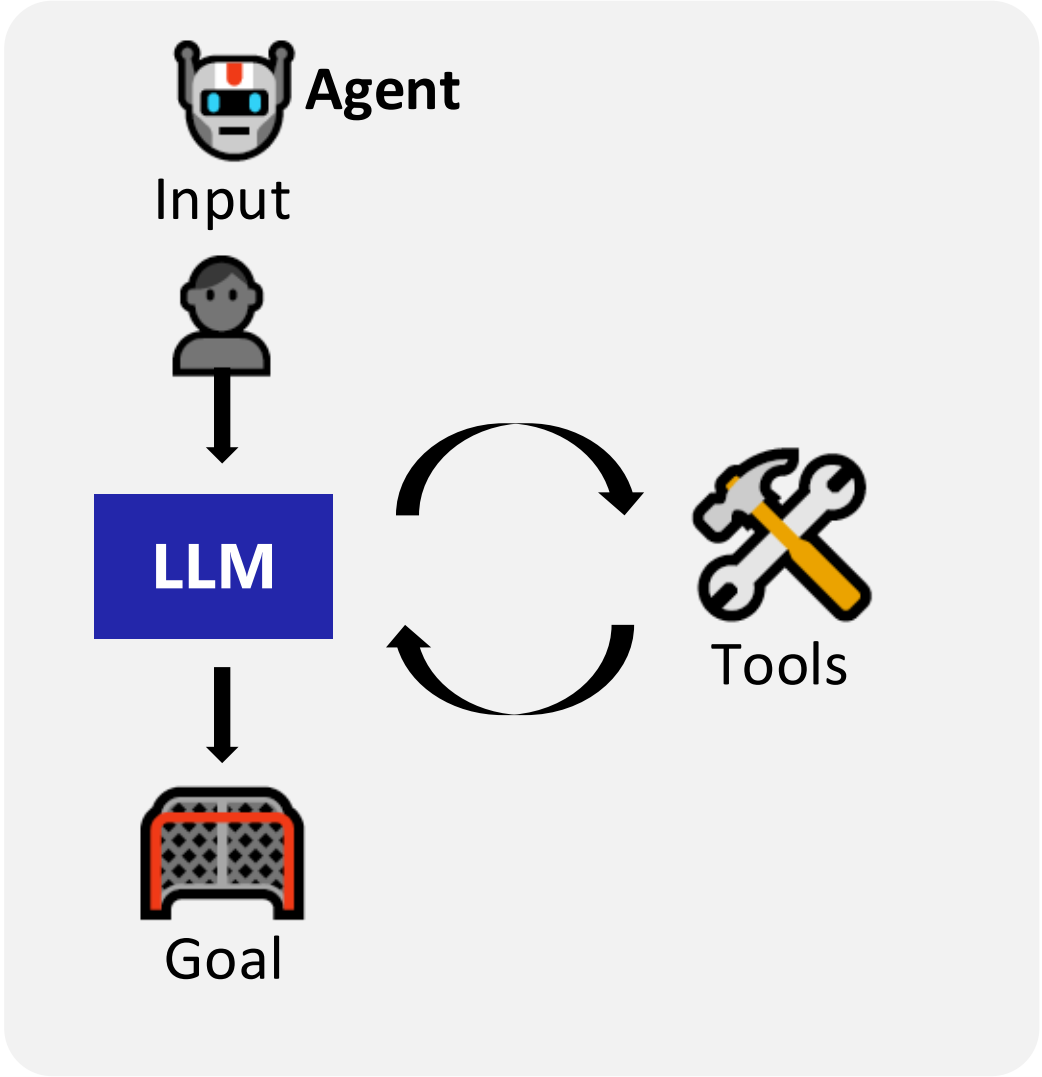
Using Work IQ as a knowledge source

```
work_source = WorkIQKnowledgeSource(
  name="workiq-knowledge-source",
  description="Contoso Work IQ knowledge source",
)
index_client.create_or_update_knowledge_source(work_source)

kb = KnowledgeBase(
  name="multisource-workiq-knowledge-base",
  knowledge_sources=[
    KnowledgeSourceReference(name="hrdocs-knowledge-source"),
    KnowledgeSourceReference(name="healthdocs-knowledge-source"),
    KnowledgeSourceReference(name="workiq-knowledge-source"),
  ],
  output_mode=OutputMode.ANSWER_SYNTHESIS,
)
index_client.create_or_update_knowledge_base(kb)
```

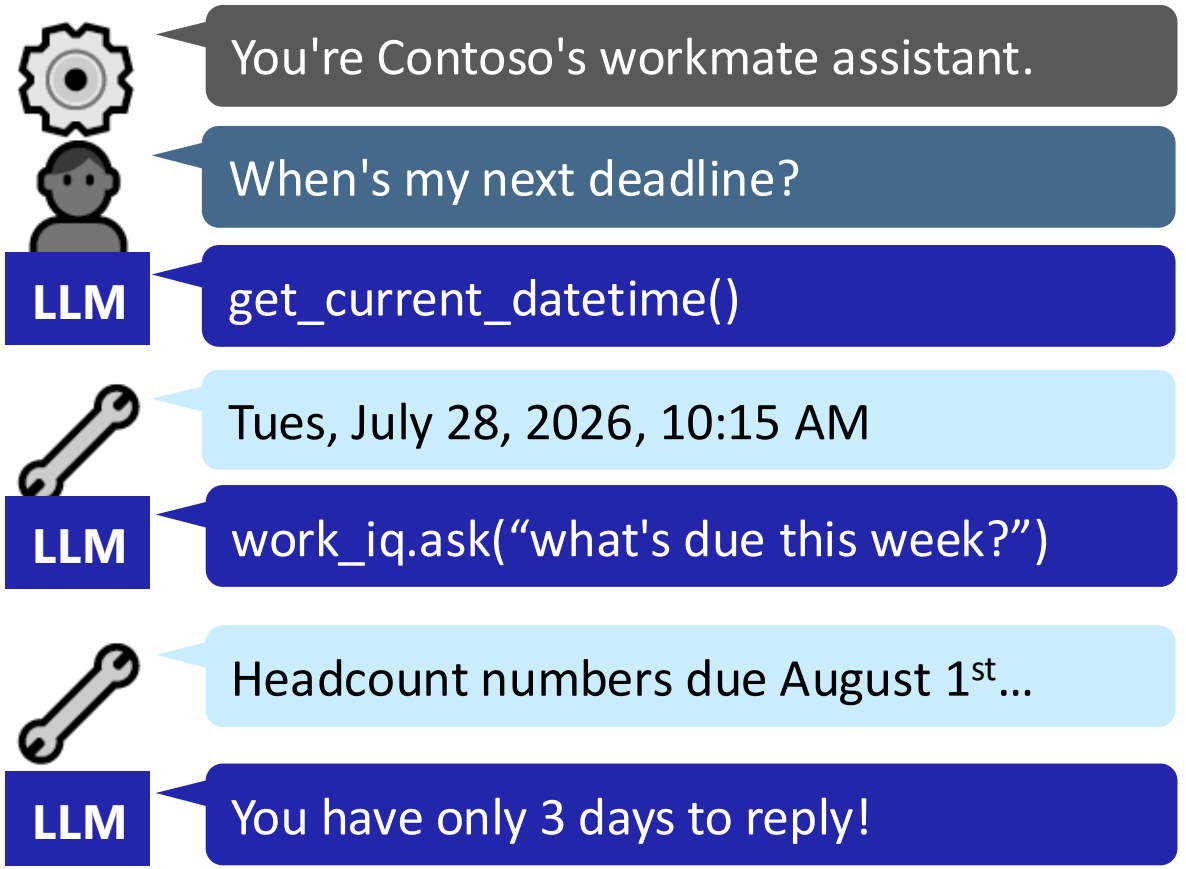
Full example: notebooks/foundryiq-workiq.ipynb

Agents + Work IQ

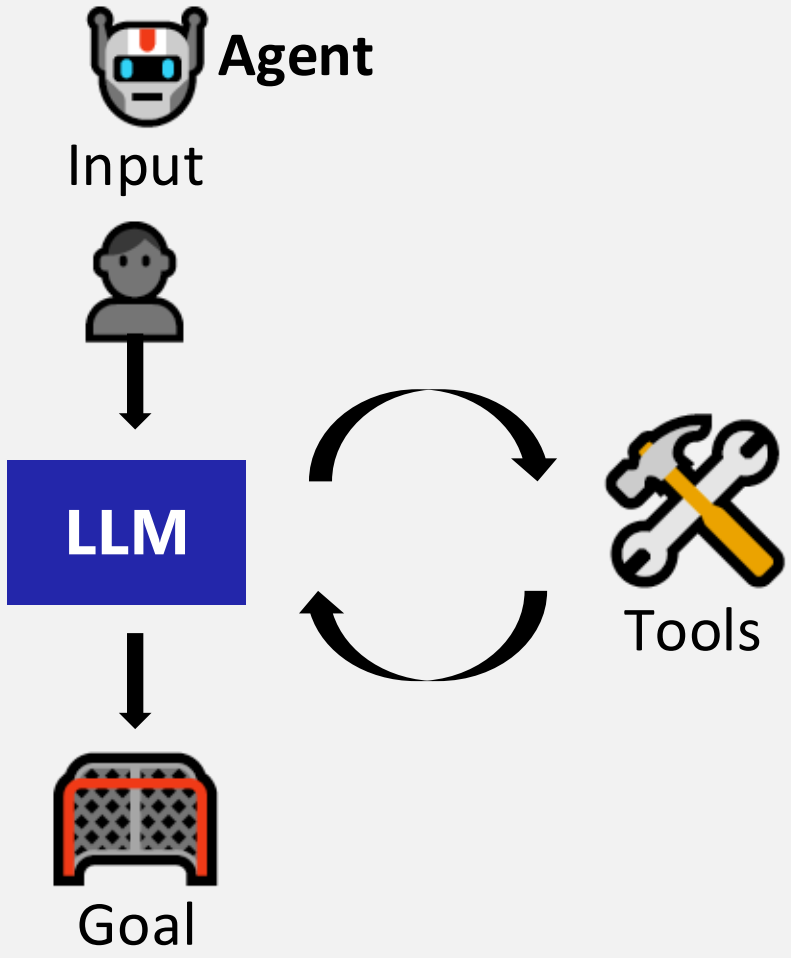
What is an agent?



An **AI agent** uses an **LLM** to run **tools** in a loop to achieve a **goal**.



Using Work IQ as a tool for AI agents



Option 1:

Point agent directly at the Work IQ MCP endpoint

- 👍 Less code to maintain
- 🤦‍♀️ Can’t customize retrieval parameters
- 🤦‍♀️ Can’t access references and activity log

Option 2:

Write a custom tool that calls Work IQ via API

- 👍 Full control over parameters and response
- 🤦‍♀️ Less portability across agents

Using Work IQ as a tool: MCP server

```
work_iq_tool = MCPStreamableHTTPTool(
  name="work-iq",
  url=work_iq_endpoint,
  http_client=work_iq_http_client,
  allowed_tools=["ask", "fetch", "get_schema", "do_action"],
  load_prompts=False)

agent = Agent(
  client=client,
  name="ContosoWorkmate",
  instructions="You are Contoso's workmate assistant.",
  tools=[get_current_date, work_iq_tool],
)
```

📎 Full example: src/workmate-agent-maf/main.py

Agent 365 autopilots

What is an Agent 365 autopilot?

Own identity

A digital worker with its own M365 identity — mailbox, calendar, Teams presence

Registered once

As an agent blueprint in Foundry; governed like any identity

@mention it

You @mention it in Teams like a coworker — it works in the background

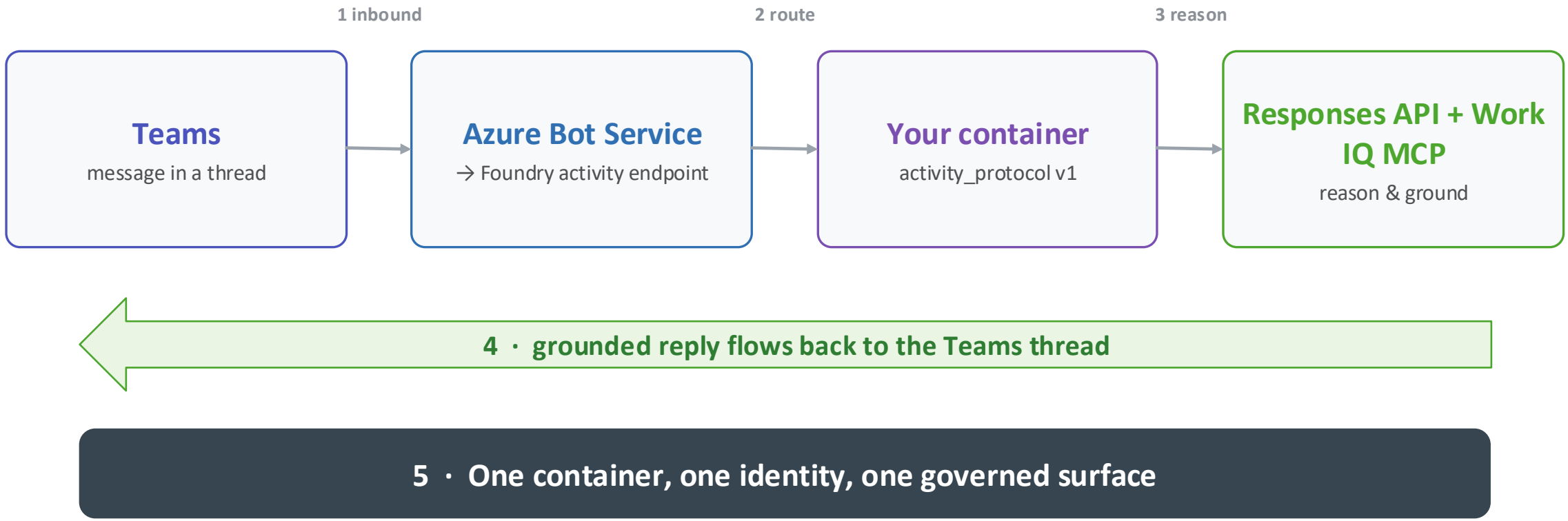
Same hosted agent

The hosted agent we built, deployed behind an activity endpoint

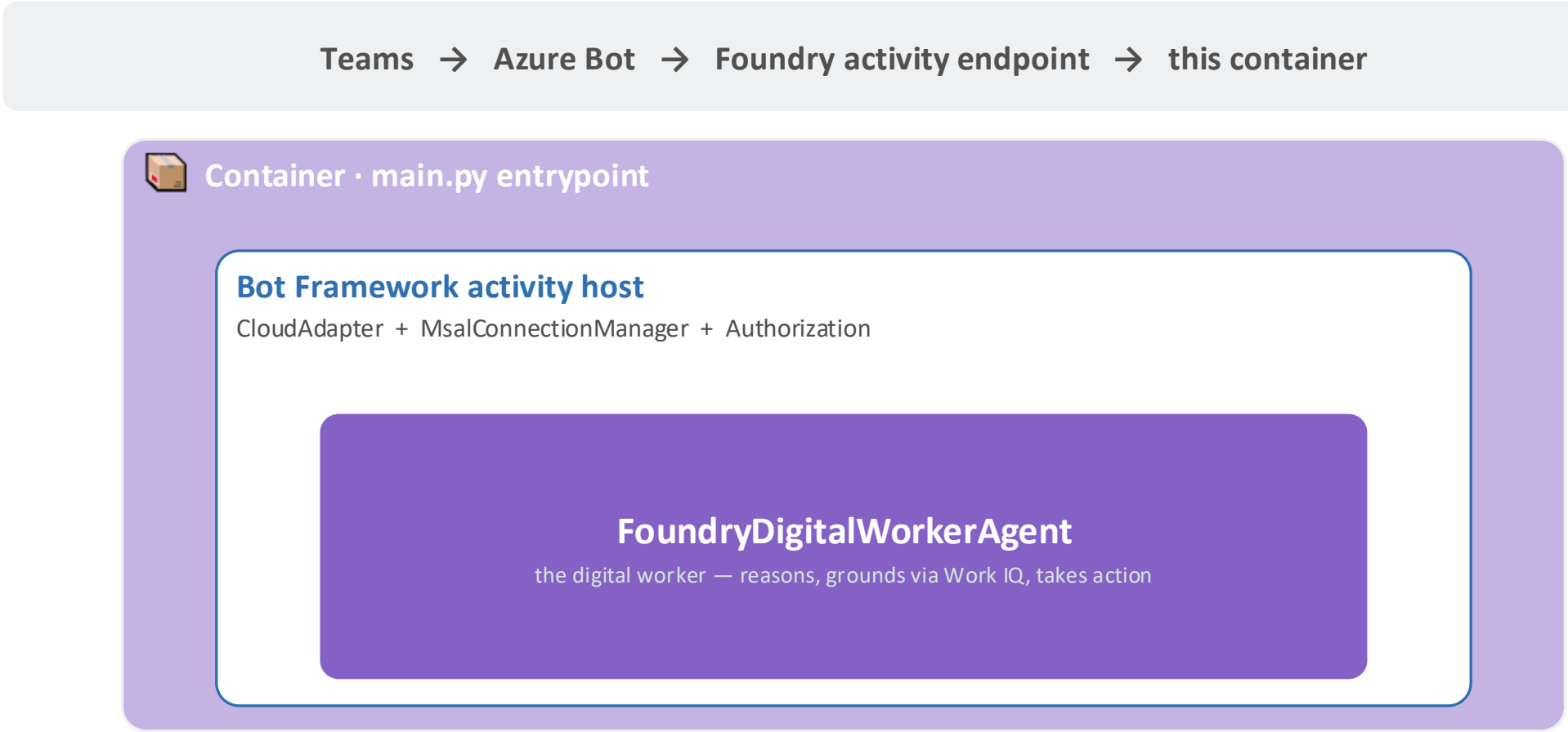
Grounded

Reasons over real work context via Work IQ

How the autopilot is wired



Hosting the autopilot: the activity host



The autopilot acts as itself — not you

In the playground

Runs on-behalf-of YOU

the signed-in user

Uses your mailbox, your permissions, your identity

vs

In Teams

Acts as its OWN identity

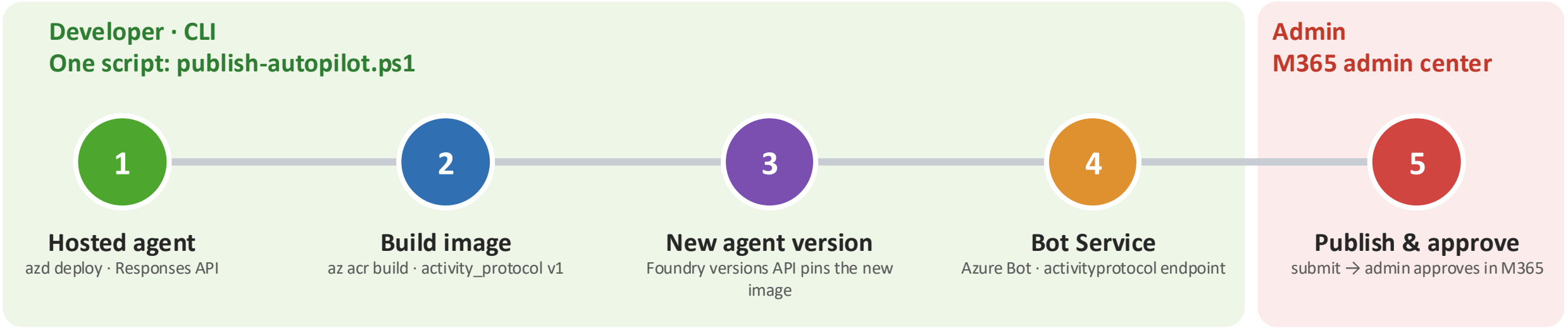
Its own mailbox & address

Consent granted to its service identity

do_action takes real action from the autopilot's M365, not yours

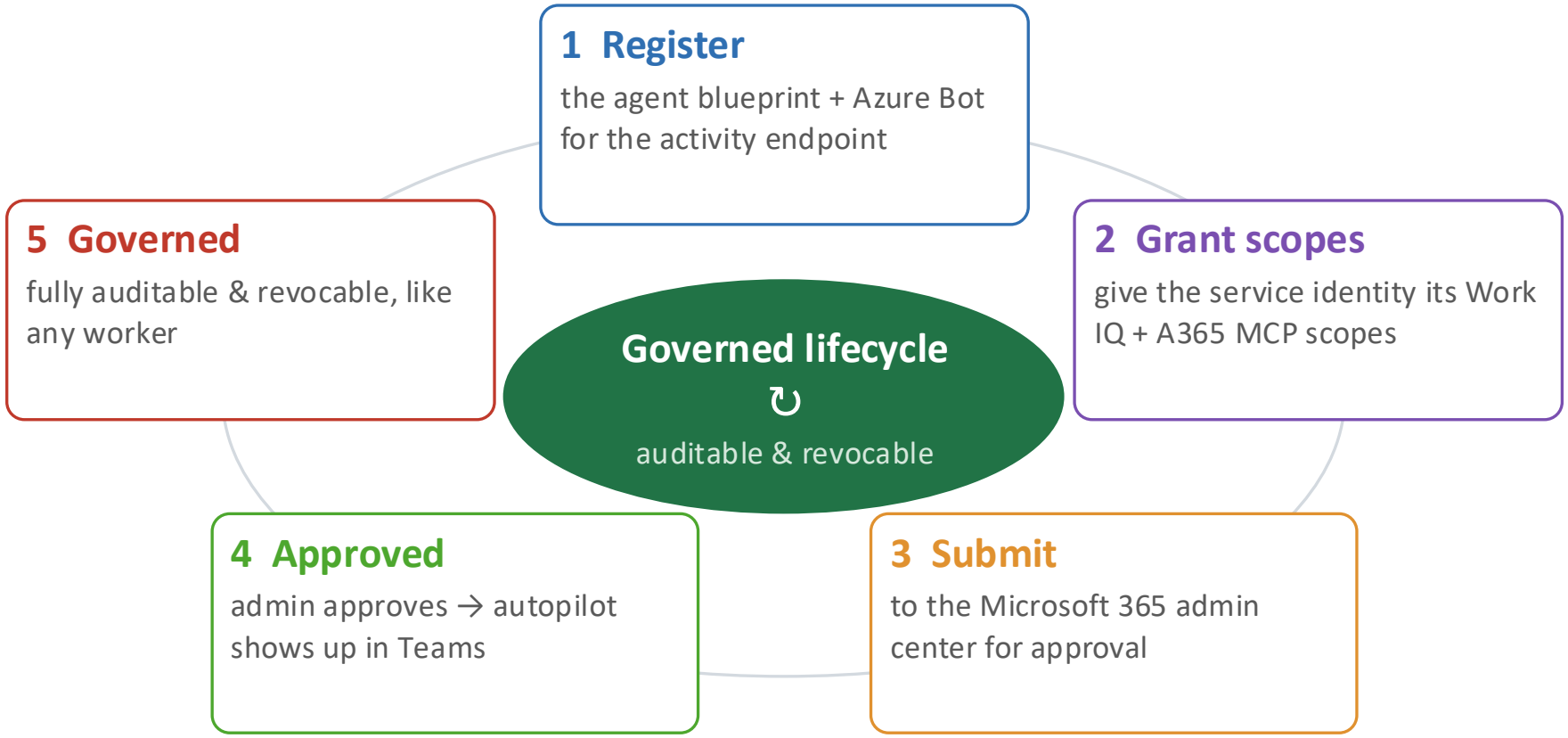
Mental model: think "digital coworker," not "an assistant signed in as me"

Making a hosted agent an autopilot



A one-click Teams publish makes an assistant. This pipeline makes an autopilot that acts as its own identity.

Publish & get approved



DEMO

Work Mate AutoPilot

Full source: src/workmate-autopilot/

Next steps

Join office hours after in Discord:
aka.ms/pythonai/oh

July 28: Foundry IQ

July 29: Work IQ

July 30: Fabric IQ

Register at aka.ms/IQDeepDivePython/series

Microsoft Reactor

JOIN US LIVE

Microsoft IQ Live

Bi-weekly episodes on Microsoft Reactor, from Aug 6 to Nov 12, 2026.

Register → aka.ms/MicrosoftIQLive

Follow Microsoft Reactor on YouTube to catch every stream.